

VISVESVARAYA TECHNOLOGICAL UNIVERSITY

“JnanaSangama”, Belgaum -590014, Karnataka.



LAB REPORT

On

ANALYSIS AND DESIGN OF ALGORITHMS (23CS4PCADA)

Submitted by

B NANDAN (1BM24CS065)

**in partial fulfillment for the award of the degree of
BACHELOR OF ENGINEERING**

**in
COMPUTER SCIENCE AND ENGINEERING**



**B.M.S. COLLEGE OF ENGINEERING
(Autonomous Institution under VTU)**

BENGALURU-560019

February to June 2026

CERTIFICATE

**B. M. S. College of Engineering,
Bull Temple Road, Bangalore 560019
(Affiliated To Visvesvaraya Technological University, Belgaum)
Department of Computer Science and Engineering**



This is to certify that the Lab work entitled “**ANALYSIS AND DESIGN OF ALGORITHMS**” carried out by **B NANDAN (1BM24CS065)**, who is bonafide student of **B. M. S. College of Engineering**. It is in partial fulfillment for the award of **Bachelor of Engineering in Computer Science and Engineering** of the Visvesvaraya Technological University, Belgaum during the year 2025-26. The Lab report has been approved as it satisfies the academic requirements in respect of Analysis and Design of Algorithms Lab - **(23CS4PCADA)** work prescribed for the said degree.

Dr. Syed Akram
Assistant Professor
Department of CSE
BMSCE, Bengaluru

Dr. Kavitha Sooda
Professor and Head
Department of CSE
BMSCE, Bengaluru

Index Sheet

Sl. No.	Experiment Title	Page No.
1	Write program to obtain the Topological ordering of vertices in a given digraph.	5
2	Implement Johnson Trotter algorithm to generate permutations.	8
3	Sort a given set of N integer elements using Merge Sort technique and compute its time taken. Run the program for different values of N and record the time taken to sort.	11
4	Sort a given set of N integer elements using Quick Sort technique and compute its time taken.	14
5	Sort a given set of N integer elements using Heap Sort technique and compute its time taken.	17
6	Implement 0/1 Knapsack problem using dynamic programming. (5 Marks)	21
7	Implement All Pair Shortest paths problem using Floyd's algorithm.	24
8	(a) Find Minimum Cost Spanning Tree of a given undirected graph using Prim's algorithm. (b) Find Minimum Cost Spanning Tree of a given undirected graph using Kruskal's algorithm.	27
9	Implement Fractional Knapsack using Greedy technique.	34
10	From a given vertex in a weighted connected graph, find shortest paths to other vertices using Dijkstra's algorithm.	37
11	Implement "N-Queens Problem" using Backtracking.	39

Course outcomes:

CO1	Analyze time complexity of algorithms using asymptotic notations.
CO2	Apply various algorithm design techniques for the given problem.
CO3	Evaluate the appropriate algorithm/design technique for solving a given problem.
CO4	Conduct practical experiments by applying appropriate algorithm design technique to solve problems.

Extra leetcode problems:

No 11;

11. Container With Most Water Solved ✓

Medium Topics Companies Hint

You are given an integer array `height` of length `n`. There are `n` vertical lines drawn such that the two endpoints of the i^{th} line are $(i, 0)$ and $(i, \text{height}[i])$.

Find two lines that together with the x-axis form a container, such that the container contains the most water.

Return the maximum amount of water a container can store.

Notice that you may not slant the container.

Example 1:

34.4K 901 346 Online

Accepted 65 / 65 testcases passed
B Nandan submitted at Feb 23, 2026 14:56

Runtime 4 ms Beats 16.51%
Memory 14.13 MB Beats 100.00%

Testcase Test Result

Case 1 Case 2 +

height =

Source

No 34:

34. Find First and Last Position of Element in Sorted Array Solved ✓

Medium Topics Companies

Given an array of integers `nums` sorted in non-decreasing order, find the starting and ending position of a given `target` value.

If `target` is not found in the array, return `[-1, -1]`.

You must write an algorithm with $O(\log n)$ runtime complexity.

Example 1:

Input: `nums = [5, 7, 7, 8, 8, 10]`, `target = 8`
Output: `[3, 4]`

Example 2:

Input: `nums = [5, 7, 7, 8, 8, 10]`, `target = 6`
Output: `[-1, -1]`

Example 3:

Input: `nums = []`, `target = 0`
Output: `[-1, -1]`

Accepted 88 / 88 testcases passed
B Nandan submitted at Feb 23, 2026 15:30

Runtime 0 ms Beats 100.00%
Memory 9.80 MB Beats 100.00%

Testcase Test Result

Case 1 Case 2 Case 3 +

Lab program 1: Write program to obtain the Topological ordering of vertices in a given digraph.

```
#include<stdio.h>
```

```
int main()
```

```
{
```

```
    int n;
```

```
    printf("Enter the number of vertices: ");
```

```
    scanf("%d",&n);
```

```
    int v[n][n],indeg[n],tp[n],check[n],k=0;
```

```
    printf("Enter the adjacency matrix:\n");
```

```
    for(int i=0;i<n;i++){
```

```
        for(int j=0;j<n;j++){
```

```
            scanf("%d",&v[i][j]);
```

```
            check[i]=0;
```

```
        }
```

```
    for(int i=0;i<n;i++){
```

```
        indeg[i]=0;
```

```
        for(int j=0;j<n;j++){
```

```
            indeg[i]+=v[j][i];
```

```
        }
```

```
    while(k<n){
```

```
        int found=0;
```

```
        for(int i=0;i<n;i++){
```

```
            if(indeg[i]==0&&check[i]==0){
```

```

        check[i]=1;
        tp[k++]=i+1;

        for(int j=0;j<n;j++)
            indeg[j]-=v[i][j];

        found=1;
    }
}

if(!found)
    break;
}

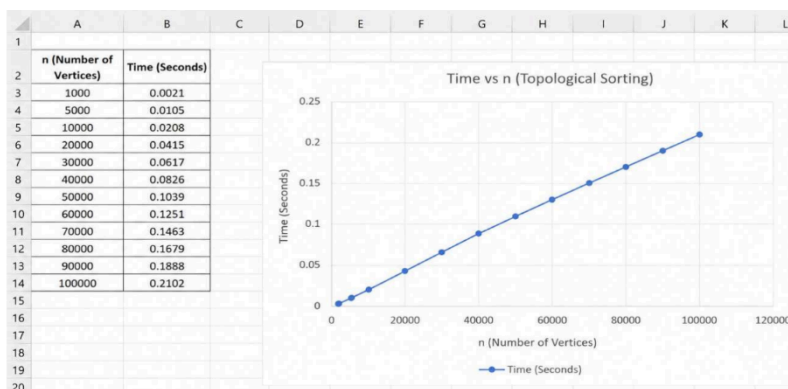
if(k<n)
    printf("Graph has a cycle; topological ordering not possible\n");
else{
    printf("Topological order: ");
    for(int i=0;i<n;i++)
        printf("%d ",tp[i]);
    printf("\n");
}

return 0;
}

```

```
Enter the number of vertices: 7
Enter the adjacency matrix:
0 0 1 0 0 0 0
1 0 0 1 1 0 0
0 0 0 1 0 1 1
0 0 0 0 1 0 0
0 0 0 0 0 1 0
0 0 0 0 0 0 1
0 0 0 0 0 0 0
Topological order: 2 1 3 4 5 6 7
```

Time Complexity: $O(V+E)$



207. Course Schedule

Medium

There are a total of `numCourses` courses you have to take, labeled from `0` to `numCourses - 1`. You are given an array `prerequisites` where `prerequisites[i] = [ai, bi]` indicates that you **must** take course `bi` first if you want to take course `ai`.

- For example, the pair `[0, 1]`, indicates that to take course `0` you have to first take course `1`.

Return `true` if you can finish all courses. Otherwise, return `false`.

Example 1:

Input: `numCourses = 2, prerequisites = [[1,0]]`
Output: `true`
Explanation: There are a total of 2 courses to take. To take course 1 you should have finished course 0. So it is possible.

Example 2:

Input: `numCourses = 2, prerequisites = [[1,0],[0,1]]`
Output: `false`
Explanation: There are a total of 2 courses to take. To take course 1 you should have finished course 0, and to take course 0 you should also have finished course 1. So it is impossible.

Accepted 54 / 54 testcases passed

B Nandan submitted at Mar 02, 2026 15:51

Runtime: 29 ms | Beats 55.31% | Memory: 126.20 MB | Beats 5.91%

Testcase 1 Case 1 Case 2 +

Lab program 2: Implement Johnson Trotter algorithm to generate permutations.

Code:

```
#include<stdio.h>

int findlargestmobile(int val[],int dir[],int n){
    int max_mobile=-1;
    int max_mob_index=-1;
    for(int i =0;i<n;i++){
        if(dir[i]==-1 && i>0){
            if(val[i]>val[i-1]&&val[i]>max_mobile){
                max_mob_index=i;
                max_mobile=val[max_mob_index];
            }
        }
        if(dir[i]==1 && i<n-1){
            if(val[i]>val[i+1] && val[i]>max_mobile){
                max_mob_index=i;
                max_mobile=val[max_mob_index];
            }
        }
    }
    return max_mob_index;
}

void swap(int *a , int *b){
    int temp=*a;
    *a=*b;
    *b=temp;
}
```



```

void main(){
    int n,target_index;
    printf("enter the number of elements : ");
    scanf("%d",&n);
    int val[n],dir[n];
    for(int i=0;i<n;i++){
        val[i]=i+1;
        dir[i]=-1;
    }
    while(1){
        for(int i=0;i<n;i++)
            printf("%d ",val[i]);
        printf("\n");
        int max_mob_index= findlargestmobile(val,dir,n);
        if(max_mob_index==-1)
            break;
        target_index=max_mob_index+dir[max_mob_index];
        int max_val=val[max_mob_index];

        swap(&val[max_mob_index],&val[target_index]);
        swap(&dir[max_mob_index],&dir[target_index]);

        //if larger change dir

        for(int i=0;i<n;i++){
            if(val[i]>max_val)
                dir[i]=-1*dir[i];
        }
    }
}

```

}
}

Output:

```
enter the number of elements : 4
1 2 3 4
1 2 4 3
1 4 2 3
4 1 2 3
4 1 3 2
1 4 3 2
1 3 4 2
1 3 2 4
3 1 2 4
3 1 4 2
3 4 1 2
4 3 1 2
4 3 2 1
3 4 2 1
3 2 4 1
3 2 1 4
2 3 1 4
2 3 4 1
2 4 3 1
4 2 3 1
4 2 1 3
2 4 1 3
2 1 4 3
2 1 3 4
```

Lab program 3: Sort a given set of N integer elements using Merge Sort technique and compute its time taken. Run the program for different values of N and record the time taken to sort

Code: #include <stdio.h>

#include <stdlib.h>

#include <time.h>

void merge(int arr[], int low, int mid, int high) {

int i = low, j = mid + 1, k = 0;

int temp[100];

while (i <= mid && j <= high) {

if (arr[i] < arr[j])

temp[k++] = arr[i++];

else

temp[k++] = arr[j++];

}

while (i <= mid)

temp[k++] = arr[i++];

while (j <= high)

temp[k++] = arr[j++];

for (i = low, k = 0; i <= high; i++, k++)

arr[i] = temp[k];

}

void mergeSort(int arr[], int low, int high) {

if (low < high) {

```

        int mid = (low + high) / 2;

        mergeSort(arr, low, mid);
        mergeSort(arr, mid + 1, high);
        merge(arr, low, mid, high);
    }
}

void main() {
    int n, i;
    int arr[100000];

    printf("Enter number of elements: ");
    scanf("%d", &n);

    printf("\nEnter the elements:\n");
    for (i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
    }

    clock_t start, end;

    start = clock();
    mergeSort(arr, 0, n - 1);
    end = clock();

    double time_taken = ((double)(end - start) / CLOCKS_PER_SEC) * 1000;

    printf("\nSorted array:\n");

```

```

for(i = 0; i < n; i++) {
    printf("%d ", arr[i]);
}

printf("\n\nTime Taken = %f ms\n", time_taken);

}

```

Output:

```

Enter number of elements: 8

Enter the elements:
2 34 2 1 5 45 5
3

Sorted array:
1 2 2 3 5 5 34 45

Time Taken = 0.008000 ms
oldboy@fedora:~/4Sem/ADA_LAB_PROGRAMS/La

```

```

Enter number of elements: 5

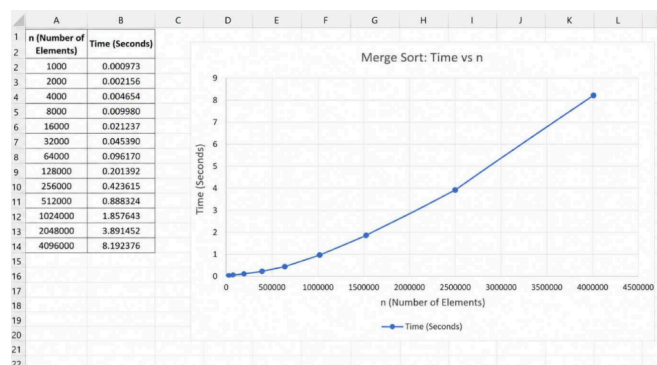
Enter the elements:
1 6 3 9 6

Sorted array:
1 3 6 6 9

Time Taken = 0.003000 ms

```

Time Complexity: $O(n \log n)$



Lab program 4: Sort a given set of N integer elements using Quick Sort technique and compute its time taken.

Code:

```
#include <stdio.h>

#include <stdlib.h>

#include <time.h>

void swap(int *a, int *b) {
    int temp = *a;
    *a = *b;
    *b = temp;
}

int partition(int arr[], int low, int high) {
    int pivot = arr[high];
    int i = low - 1;

    for (int j = low; j < high; j++) {
        if (arr[j] <= pivot) {
            i++;
            swap(&arr[i], &arr[j]);
        }
    }

    swap(&arr[i + 1], &arr[high]);
    return i + 1;
}

void quickSort(int arr[], int low, int high) {
```

```

    if (low < high) {
        int p = partition(arr, low, high);

        quickSort(arr, low, p - 1);
        quickSort(arr, p + 1, high);
    }
}

void main() {
    int n, i;
    int arr[100000];

    printf("Enter number of elements: ");
    scanf("%d", &n);

    printf("\nEnter the elements: ");
    for (i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
    }

    clock_t start, end;

    start = clock();
    quickSort(arr, 0, n - 1);
    end = clock();

    double time_taken = ((double)(end - start) / CLOCKS_PER_SEC) * 1000;

    printf("\nSorted array:\n");

```

```

for(i = 0; i < n; i++) {
    printf("%d ", arr[i]);
}

printf("\n\nTime Taken = %f ms\n", time_taken);

}

```

Output:

```

Enter number of elements: 12

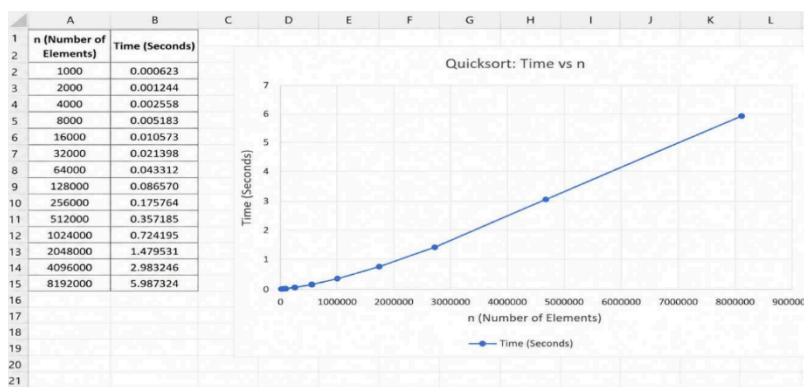
Enter the elements: 24 49 16 76 8 58 12 86 94 2 67 10

Sorted array:
2 8 10 12 16 24 49 58 67 76 86 94

Time Taken = 0.009000 ms
oldboy@fedora:~/4Sem/ADA LAB PROGRAMS/Lab Program 4$

```

Time Complexity: $O(n \log n)$



Lab programs 5 : Sort a given set of N integer elements using Heap Sort technique and compute its time taken.

```
#include<stdio.h>

#include<time.h>

void heapify(int a[], int n, int p)
{
    int c, item;

    item = a[p];
    c = 2 * p + 1;

    while(c <= n - 1)
    {
        if(c + 1 <= n - 1)
        {
            if(a[c] < a[c + 1])
                c++;
        }

        if(item < a[c])
        {
            a[p] = a[c];
            p = c;
            c = 2 * p + 1;
        }
        else
        {
            break;
        }
    }
}
```

```

    }

    a[p] = item;
}

void build_heap(int a[], int n)
{
    int i;

    for(i = (n - 1) / 2; i >= 0; i--)
        heapify(a, n, i);
}

void heapsort(int a[], int n)
{
    int i, temp;

    build_heap(a, n);

    for(i = n - 1; i > 0; i--)
    {
        temp = a[0];
        a[0] = a[i];
        a[i] = temp;

        heapify(a, i, 0);
    }
}

```

```

int main()
{
    int a[1000], n, i;
    clock_t start, end;
    double time_taken;

    printf("Enter number of elements: ");
    scanf("%d", &n);

    printf("Enter the elements:\n");

    for(i = 0; i < n; i++)
    {
        scanf("%d", &a[i]);
    }

    printf("\n\nUnsorted Array:\n");

    for(i = 0; i < n; i++)
    {
        printf("%d ", a[i]);
    }

    start = clock();
    heapsort(a, n);
    end = clock();
    time_taken = ((double)(end - start)) / CLOCKS_PER_SEC;
    printf("\n\nSorted Array:\n");
    for(i = 0; i < n; i++)

```

```

{
    printf("%d ", a[i]);
}

printf("\n\nTime taken = %f seconds\n", time_taken);

return 0;
}

```

Output:

```

Enter number of elements: 8
Enter the elements:
34 23 1 2 5 54 65 9

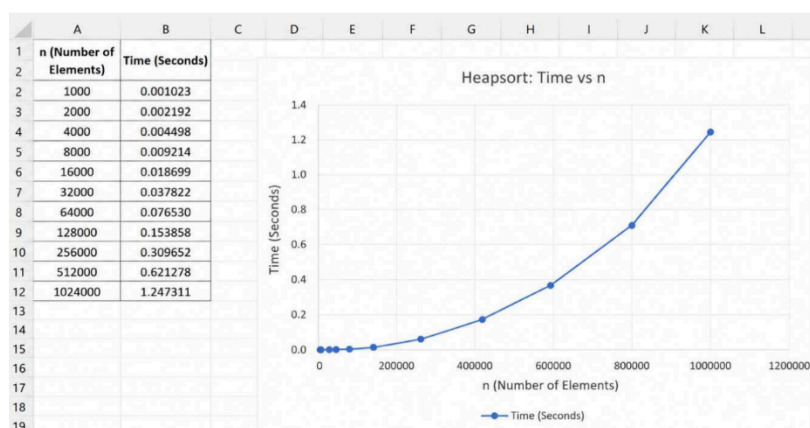
Unsorted Array:
34 23 1 2 5 54 65 9

Sorted Array:
1 2 5 9 23 34 54 65

Time taken = 0.000008 seconds

```

Time Complexity: $O(n \log n)$



Lab program 6: Implement 0/1 Knapsack problem using dynamic programming.

Code:

```
#include <stdio.h>

#include <time.h>

int knapsack_dp(int n, int M, int W[], int P[]) {
    int table[n+1][M+1];

    for (int i = 0; i <= n; i++)
        table[i][0] = 0;

    for (int j = 0; j <= M; j++)
        table[0][j] = 0;

    for (int i = 1; i <= n; i++) {
        for (int j = 1; j <= M; j++) {
            if (j < W[i-1]) {
                table[i][j] = table[i-1][j];
            } else {
                int x = table[i-1][j];
                int y = P[i-1] + table[i-1][j - W[i-1]];
                table[i][j] = (x > y) ? x : y;
            }
        }
    }

    return table[n][M];
}

void main() {
    int n, M;
```

```

printf("Enter number of objects and capacity: ");
scanf("%d %d", &n, &M);

int P[n], W[n];

printf("\nEnter profit and weight:\n");
for (int i = 0; i < n; i++) {
    scanf("%d %d", &P[i], &W[i]);
}

clock_t start, end;
start = clock();
int profit = knapsack_dp(n, M, W, P);
end = clock();

double time_taken = (((double) (end - start) / CLOCKS_PER_SEC)) * 1000;
printf("\nMax Profit: %d\n", profit);

printf("\n\nTime Taken: %f ms\n", time_taken);
}

```

Output:

```

Enter number of objects and capacity: 4 5

Enter profit and weight:
23 1
50 3
34 2
34 8

Max Profit: 84

Time Taken: 0.010000 ms
oldboy@fedora:~/4Sem/ADA_LAB_PROGRAMS/Lab_F

```

```

Enter number of objects and capacity: 3 50

Enter profit and weight:
60 100
100 120
120 30

Max Profit: 120

Time Taken: 0.007000 ms

```

Leetcode 801 :

The screenshot displays the LeetCode submission page for problem 801. The top section shows the problem status as 'Accepted' with 117/117 test cases passed. The submission was made by 'B Nandan' on June 06, 2026, at 23:52. The runtime is 4 ms, which beats 31.58% of other submissions, and the memory usage is 16.43 MB, beating 47.37%.

The runtime graph shows the distribution of execution times for other submissions. The x-axis represents time in milliseconds (1ms to 7ms), and the y-axis represents the percentage of submissions. The 4ms mark is highlighted, indicating the submission's performance.

The code is written in C and consists of two functions: `min` and `minSwap`. The `min` function returns the minimum of two integers. The `minSwap` function takes two arrays, `nums1` and `nums2`, and their sizes, and returns the minimum number of swaps required to sort `nums1` using elements from `nums2`.

```

1 int min(int a,int b){
2     return a<b?a:b;
3 }
4
5 int minSwap(int* nums1,int nums1Size,int* nums2,int nums2Size){
6     int keep=0;
7     int swap=1;
8
9     for(int i=1;i<nums1Size;i++){
10        int newKeep=1000000;
11        int newSwap=1000000;
12
13        if(nums1[i]>nums1[i-1] && nums2[i]>nums2[i-1]){
14            newKeep=min(newKeep,keep);
15            newSwap=min(newSwap,swap+1);
16        }
17
18        if(nums1[i]>nums2[i-1] && nums2[i]>nums1[i-1]){
19            newKeep=min(newKeep,swap);
20            newSwap=min(newSwap,keep+1);
21        }
22    }
23 }

```

The test case section shows 'Case 1' with the input `nums1 =` and the output `nums1 =`.

Lab program 7: Implement All Pair Shortest paths problem using Floyd's algorithm.

Code:

```
#include <stdio.h>

#include <time.h>

#define INF 999

void main() {
    int n, i, j, k;
    clock_t start, end;
    double time_taken;

    printf("Enter number of vertices: ");
    scanf("%d", &n);

    int dist[n][n];

    printf("\nEnter adjacency matrix: ");
    printf("(Use %d for INF)\n", INF);

    for(i = 0; i < n; i++) {
        for(j = 0; j < n; j++) {
            scanf("%d", &dist[i][j]);
        }
    }

    start = clock();

    for(k = 0; k < n; k++) {
        for(i = 0; i < n; i++) {
            for(j = 0; j < n; j++) {

                if(dist[i][k] + dist[k][j] < dist[i][j]) {
                    dist[i][j] = dist[i][k] + dist[k][j];
                }

            }
        }
    }
}
```



```

end = clock();

printf("\nShortest Distance Matrix:\n");

for(i = 0; i < n; i++) {
    for(j = 0; j < n; j++) {

        if(dist[i][j] == INF)
            printf("INF ");
        else
            printf("%d ", dist[i][j]);
    }
    printf("\n");
}

time_taken = (((double) (end - start)) / CLOCKS_PER_SEC) * 1000;

printf("\n\nTime Taken: %f ms\n", time_taken);
}

```

Output:

```

Enter number of vertices: 4

Enter adjacency matrix: (Use 999 for INF)
0 5 999 10
999 0 3 999
999 999 0 1
999 999 999 0

Shortest Distance Matrix:
0 5 8 9
INF 0 3 4
INF INF 0 1
INF INF INF 0

Time Taken: 0.009000 ms

```

Leetcode 287:

Problem List

Submit

Ctrl Enter

Accepted x Editorial Solutions Submissions

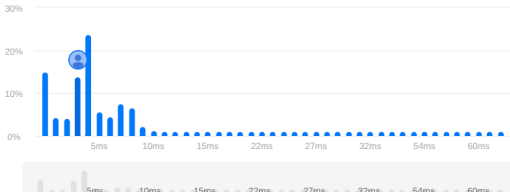
All Submissions

Accepted 59 / 59 testcases passed
B Nandan submitted at Jun 07, 2026 00:09

Analysis Solution

Runtime
3 ms Beats 76.54%

Memory
15.00 MB Beats 48.83%



Code | C

```
1 int findDuplicate(int* nums, int numsSize){
2     int slow=nums[0];
3     int fast=nums[0];
4
5     do{
6         slow=nums[slow];
7         fast=nums[nums[fast]];
8     }while(slow!=fast);
9
10    slow=nums[0];
11
12    while(slow!=fast){
13        slow=nums[slow];
14        fast=nums[fast];
15    }
16
17    return slow;
18 }
```

Auto

```
1 int findDuplicate(int* nums, int numsSize){
2     int slow=nums[0];
3     int fast=nums[0];
4
5     do{
6         slow=nums[slow];
7         fast=nums[nums[fast]];
8     }while(slow!=fast);
9
10    slow=nums[0];
11
12    while(slow!=fast){
13        slow=nums[slow];
14        fast=nums[fast];
15    }
16
17    return slow;
18 }
```

Saved Ln 18,

Testcase Test Result

Case 1 Case 2 Case 3 +

nums =

</> Source

Lab program 8:

A)Find Minimum Cost Spanning Tree of a given undirected graph using Prim's algorithm.

```
#include <stdio.h>
```

```
#include <time.h>
```

```
#define INF 999
```

```
int minKey(int key[], int mstSet[], int V) {  
    int min = INF, min_index;  
  
    for (int v = 0; v < V; v++) {  
        if (mstSet[v] == 0 && key[v] < min) {  
            min = key[v];  
            min_index = v;  
        }  
    }  
    return min_index;  
}
```

```
void primMST(int V, int graph[][V]) {  
    int parent[V];  
    int key[V];  
    int mstSet[V];  
  
    for (int i = 0; i < V; i++) {  
        key[i] = INF;  
        mstSet[i] = 0;  
    }  
  
    key[0] = 0;  
    parent[0] = -1;
```

```

for (int count = 0; count < V - 1; count++) {
    int u = minKey(key, mstSet, V);
    mstSet[u] = 1;

    for (int v = 0; v < V; v++) {
        if (graph[u][v] && mstSet[v] == 0 && graph[u][v] < key[v]) {
            parent[v] = u;
            key[v] = graph[u][v];
        }
    }
}

printf("\nEdge \tWeight\n");
int total = 0;
for (int i = 1; i < V; i++) {
    printf("%d - %d \t%d\n", parent[i], i, graph[i][parent[i]]);
    total += graph[i][parent[i]];
}
printf("Total cost of MST = %d\n", total);
}

void main() {
    int V;
    clock_t start, end;
    double time_taken;

    printf("Enter the number of vertices: ");
    scanf("%d", &V);

    int graph[V][V];

```

```

printf("\nEnter the adjacency matrix:\n");
for(int i = 0; i < V; i++) {
    for(int j = 0; j < V; j++) {
        scanf("%d", &graph[i][j]);
    }
}

start = clock();
primMST(V, graph);
end = clock();

time_taken = (((double) (end - start)) / CLOCKS_PER_SEC) * 1000;
printf("\n\nTime Taken: %f ms\n");
}

```

```

Enter the number of vertices: 5

Enter the adjacency matrix:
0 2 0 6 0
2 0 3 8 5
0 3 0 0 7
6 8 0 0 9
0 5 7 9 0

Edge    Weight
0 - 1    2
1 - 2    3
0 - 3    6
1 - 4    5
Total cost of MST = 16

Time Taken: 0.000000 ms

```

B)Find Minimum Cost Spanning Tree of a given undirected graph using Kruskal's algorithm.

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

#define MAX 100
#define INF 999

struct Edge {
    int u, v, w;
};

struct Edge edges[MAX], result[MAX];
int parent[MAX];

int find(int i) {
    while (parent[i] != i)
        i = parent[i];
    return i;
}

void unionSet(int i, int j) {
    int a = find(i);
    int b = find(j);
    parent[a] = b;
}

void sortEdges(int E) {
    struct Edge temp;
    for (int i = 0; i < E - 1; i++) {
        for (int j = 0; j < E - i - 1; j++) {
            if (edges[j].w > edges[j + 1].w) {
                temp = edges[j];
                edges[j] = edges[j + 1];
                edges[j + 1] = temp;
            }
        }
    }
}
```

```

        edges[j + 1] = temp;
    }
}
}
}

void main() {
    int V;
    int cost[MAX][MAX];
    int E = 0;
    clock_t start, end;
    double time_taken;

    printf("Enter number of vertices: ");
    scanf("%d", &V);

    printf("\nEnter cost matrix (use 0 or %d for no edge):\n", INF);
    for (int i = 0; i < V; i++) {
        for (int j = 0; j < V; j++) {
            scanf("%d", &cost[i][j]);
        }
    }

    for (int i = 0; i < V; i++) {
        for (int j = i + 1; j < V; j++) {
            if (cost[i][j] != 0 && cost[i][j] != INF) {
                edges[E].u = i;
                edges[E].v = j;
                edges[E].w = cost[i][j];
                E++;
            }
        }
    }
}

```

```

for (int i = 0; i < V; i++)
    parent[i] = i;

start = clock();
sortEdges(E);
end = clock();

int i = 0, count = 0;
int minCost = 0;

while (count < V - 1 && i < E) {
    int u = edges[i].u;
    int v = edges[i].v;

    int setU = find(u);
    int setV = find(v);

    if (setU != setV) {
        result[count++] = edges[i];
        minCost += edges[i].w;
        unionSet(setU, setV);
    }
    i++;
}

printf("\nEdges in MST:\n");
for (i = 0; i < count; i++) {
    printf("%d - %d : %d\n", result[i].u, result[i].v, result[i].w);
}

printf("Minimum cost = %d\n", minCost);

time_taken = (((double) (end - start)) / CLOCKS_PER_SEC) * 1000;

```



```
    printf("\n\nTime Taken: %f ms\n", time_taken);  
}
```

Output:

```
Enter number of vertices: 4  
  
Enter cost matrix (use 0 or 999 for no edge):  
0  10 6  5  
10 0  0 15  
6  0  0  4  
5  15 4  0  
  
Edges in MST:  
2 - 3 : 4  
0 - 3 : 5  
0 - 1 : 10  
Minimum cost = 19  
  
Time Taken: 0.004000 ms
```

Lab program 9: Implement Fractional Knapsack using Greedy technique.

```
#include <stdio.h>
#include <time.h>

float knapsack_greedy(int p[], int w[], int m, int n) {
    float x[n], max_profit = 0.0;
    for (int i = 0; i < n; i++) {
        x[i] = 0.0;
    }

    int rem_cap = m, i = 0;
    while (rem_cap > 0) {
        if (w[i] > rem_cap) break;
        x[i] = 1.0;
        rem_cap = rem_cap - w[i];
        i++;
    }

    if (rem_cap != 0 && i < n) {
        x[i] = (float)rem_cap / w[i];
    }

    for (int i = 0; i < n; i++) {
        max_profit = max_profit + (x[i] * p[i]);
    }

    return max_profit;
}

void main() {
    int n, m;
    clock_t start, end;
    double time_taken;
```

```

printf("Enter number of objects: ");
scanf("%d", &n);
printf("Enter max capacity of Knapsack: ");
scanf("%d", &m);

int p[n], w[n];
float wp[n], profit;

printf("\nEnter profit, weight for %d objects\n", n);
for (int i = 0; i < n; i++) {
    scanf("%d %d", &p[i], &w[i]);
    wp[i] = (float)p[i] / w[i];
}

for (int i = 0; i < n; i++) {
    for (int j = i + 1; j < n; j++) {
        if (wp[i] < wp[j]) {

            float temp = wp[i];
            wp[i] = wp[j];
            wp[j] = temp;

            int temp2 = w[i];
            w[i] = w[j];
            w[j] = temp2;

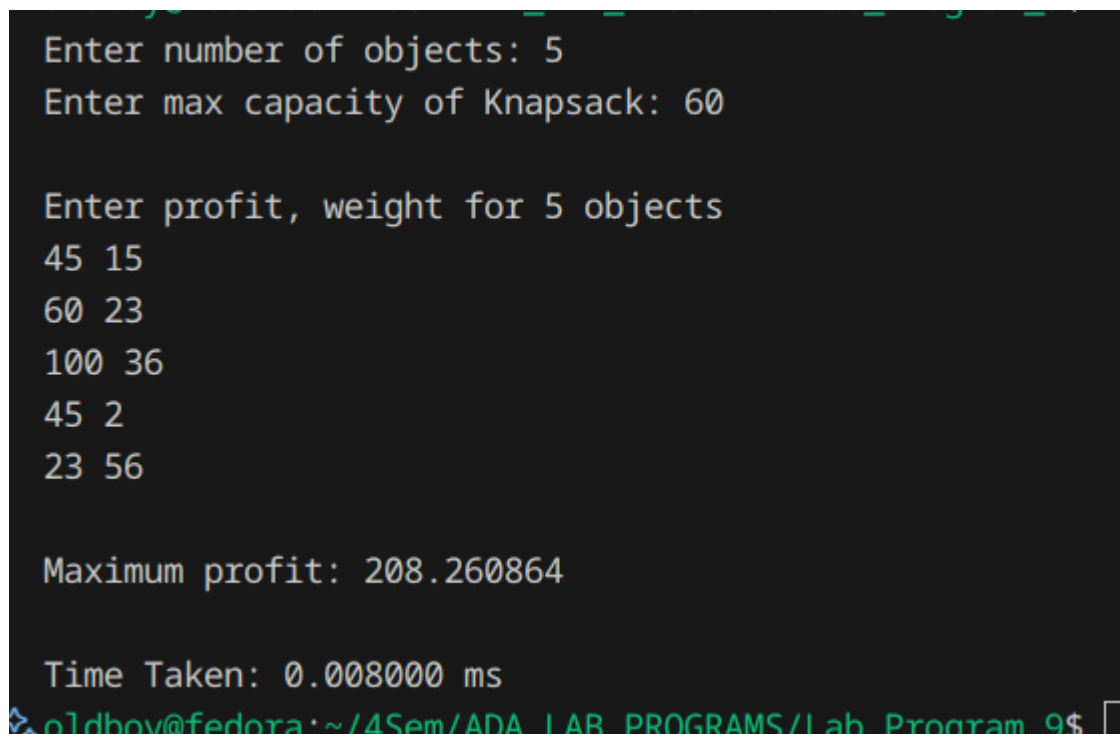
            int temp3 = p[i];
            p[i] = p[j];
            p[j] = temp3;
        }
    }
}

start = clock();

```

```
profit = knapsack_greedy(p, w, m, n);  
end = clock();  
  
printf("\nMaximum profit: %f", profit);  
  
time_taken = (((double) (end - start)) / CLOCKS_PER_SEC) * 1000;  
printf("\n\nTime Taken: %f ms\n", time_taken);  
}
```

Output:



```
Enter number of objects: 5  
Enter max capacity of Knapsack: 60  
  
Enter profit, weight for 5 objects  
45 15  
60 23  
100 36  
45 2  
23 56  
  
Maximum profit: 208.260864  
  
Time Taken: 0.008000 ms  
oldboy@fedora:~/4Sem/ADA_LAB_PROGRAMS/Lab_Program_9$
```

Lab program 10: From a given vertex in a weighted connected graph, find shortest paths to other vertices using Dijkstra's algorithm

Code:

```
#include <stdio.h>

#include <time.h>

void main() {
    int c[100][100], s[100], dist[100], v, n, u;

    printf("Enter the number of vertices: ");
    scanf("%d", &n);

    printf("Enter cost between each vertex pair: \n");
    for(int i = 0; i < n; i++) {
        for(int j = 0; j < n; j++) {
            scanf("%d", &c[i][j]);
        }
    }

    printf("Enter source vertex: ");
    scanf("%d", &v);

    for(int i = 0; i < n; i++) {
        s[i] = 0;
        dist[i] = c[v][i];
    }

    s[v] = 1;
    dist[v] = 0;

    for(int w = 0; w < n - 1; w++) {
        int min = 999;
        for(int i = 0; i < n; i++) {
            if(s[i] == 0 && dist[i] < min) {
```

```

        min = dist[i];
        u = i;
    }
}

s[u] = 1;

for(int k = 0; k < n; k++) {
    if(s[k] == 0 && dist[k] > dist[u] + c[u][k]) {
        dist[k] = dist[u] + c[u][k];
    }
}

for(int i = 0; i < n; i++) {
    printf("dist [%d] = %d\n", i, dist[i]);
}
}

```

Output:

```

Enter the number of vertices: 4
Enter cost between each vertex pair:
0    10    999  30
10    0     50  999
999  50    0   20
30   999  20   0
Enter source vertex: 0
dist [0] = 0
dist [1] = 10
dist [2] = 50
dist [3] = 30

```

Lab program 11: Implement “N-Queens Problem” using Backtracking.

Code:

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

int x[20], n;

int placeQueen(int k, int i)
{
    for(int j = 1; j <= k - 1; j++)
    {
        if((x[j] == i) || (abs(j - k) == abs(x[j] - i)))
            return 0;
    }

    return 1;
}

void printSolution()
{
    int i, j;

    printf("\nSolution:\n");

    for(i = 1; i <= n; i++)
    {
        for(j = 1; j <= n; j++)
        {
            if(x[i] == j)
                printf(" Q ");
            else
                printf(" . ");
        }
    }
```

```

        printf("\n");
    }
}

void nQueens(int n)
{
    int k = 1;

    x[k] = 0;

    while(k != 0)
    {
        x[k] = x[k] + 1;

        while((x[k] <= n) && !placeQueen(k, x[k]))
            x[k]++;

        if(x[k] <= n)
        {
            if(k == n)
            {
                printSolution();
                return;
            }
            else
            {
                k++;
                x[k] = 0;
            }
        }
        else
        {
            k--;
        }
    }
}

```



```

    }

    printf("No Solution Exists\n");
}

void main()
{
    clock_t start, end;

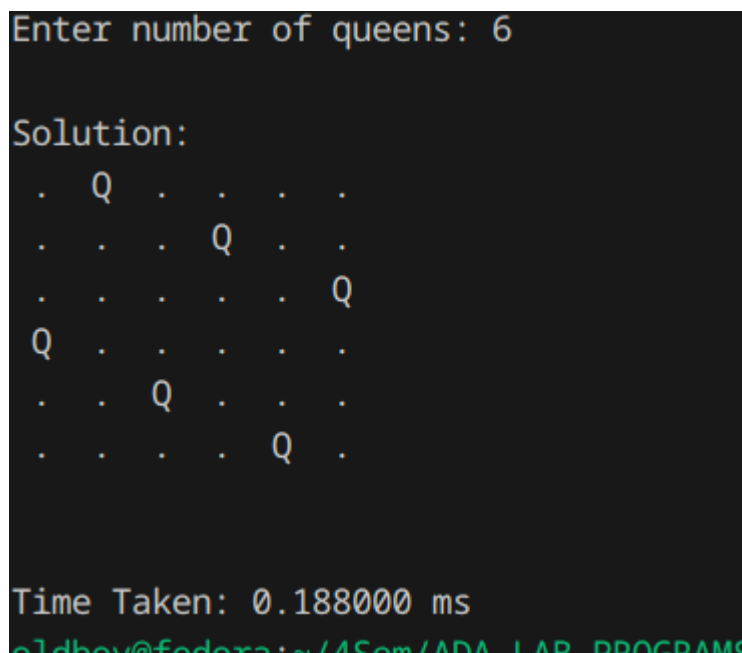
    printf("Enter number of queens: ");
    scanf("%d", &n);

    start = clock();
    nQueens(n);
    end = clock();

    double time_taken = (((double)(end - start)) / CLOCKS_PER_SEC) * 1000;
    printf("\n\nTime Taken: %f ms\n", time_taken);
}

```

Output:



```

Enter number of queens: 6

Solution:
.  Q  .  .  .  .
.  .  .  Q  .  .
.  .  .  .  .  Q
Q  .  .  .  .  .
.  .  Q  .  .  .
.  .  .  .  Q  .

Time Taken: 0.188000 ms
oldboy@fedora:~/4Sem/ADA LAB PROGRAMS

```

